COMP3028 / LECTURE 10

Optimising Server Management

Beginner's guide and step-by-step hands-on workbook

Measure a local Node.js server, explain what slows it down, and test changes with evidence.

Includes

Worked service-level calculations; six guided labs; runnable source code; expected observations; troubleshooting; interpretation exercises.

Rewritten and expanded from the supplied lecture.
English edition / November 2026

Contents

1 How to use this guide **2**

2 Concepts: from deployment to dependable service **3**

3 Worked calculations: service quality **4**

4 Lab 1: setup and your first request **5**

5 Reading the server code without getting lost **6**

6 Lab 2: metrics and controlled errors **7**

7 Lab 3: observe blocking before changing it **8**

8 Lab 4: measure a cache and its expiry **9**

9 Lab 5: two workers, separate state **10**

10 Lab 6: optional PM2 management **11**

11 Beyond the lab: queries, pools, and multiple hosts **12**

12 Troubleshooting and measurement discipline **13**

13 Evidence worksheet and final case study **14**

14 References and scope of the rewrite **15**

A Complete runnable source **16**

A.1 app.js 16

A.2 load.js 17

A.3 cluster.js 18

A.4 inspect–workers.js 18

1 How to use this guide

This guide accompanies the rewritten COMP3028 Lecture 10 slides. It explains the concepts more slowly and supplies a complete local laboratory. The core application uses Node.js's built-in HTTP module so that no framework installation is needed. The same request-handling ideas apply to Express middleware and routes.

Before you begin

You should know how to open a terminal, change folders, run a JavaScript file, and recognise a function. You do not need previous server-administration experience. An HTTP request asks a server to perform an operation; the response contains a status code and a body. In this lab, the body is JSON.

Use Node.js 22 or newer with its matching npm. The supplied code was checked with Node.js 24.22.1 in the authoring environment. Use a currently supported release for your own setup. No database, cloud account, Docker installation, or administrator privileges are required for the core lab.

Suggested time	Activity
15 minutes	Setup and first request
20 minutes	Metrics and service calculations
20 minutes	Blocking and asynchronous waiting
15 minutes	Cache experiments
20 minutes	Clustering and state
15 minutes	Optional PM2 extension
15 minutes	Interpretation and submission

Files supplied

`app.js` is the server, `load.js` sends a bounded local workload, `cluster.js` starts two workers, and `inspect-workers.js` prints worker IDs and counters. All are in the `lab` folder. The appendix includes their full source.

Learning method

Before every experiment, write a prediction. Run the commands, save the output, explain any difference, and repeat when necessary. Numerical examples labelled "illustrative" are teaching examples, not promised results for your computer. Use two terminals: A runs the server, B runs the client.

2 Concepts: from deployment to dependable service

Why deployment is only the beginning

A process can keep running while users receive errors. For example, the web server may accept connections even though its database is unavailable. Server management therefore combines process supervision with checks of real user operations. Maintenance also includes permissions, software updates, configuration, backups, and capacity planning.

Think of an application as a cafe. CPU capacity resembles available attention for work, memory resembles working space, a database resembles the order book, and a queue resembles customers waiting. Adding another cashier helps only when cashier capacity is the constraint. If the kitchen is already overloaded, more cashiers can increase the backlog.

Three distinct questions

1. **Correctness:** did the system register the right student for the right event?
2. **Availability:** could an eligible student complete registration?
3. **Performance:** how long did it take under the stated workload?

A health endpoint is a small liveness check in this lab. It confirms that this process can respond. A production readiness check may additionally assess whether an instance can accept real work, using carefully bounded dependency checks. Readiness and liveness checks serve different purposes.

A change needs a recovery path

Before changing software or configuration, know the current version and how to return to it. Test the new behaviour in a suitable environment, deploy with observation, and check important user journeys. A successful installation message is insufficient evidence. Backups need a restore exercise to establish that the data and recovery procedure actually work.

Small glossary

Term	Meaning in this lab
Process	A running program with its own memory space
PID	The operating-system identifier for a process
Endpoint	A path that the server recognises, such as `/health`
Concurrency	Number of requests allowed in progress together
Throughput	Completed requests divided by elapsed time
Latency	Time taken for an individual request
TTL	How long a cached value remains valid

3 Worked calculations: service quality

Example A: time-based availability

Assume a 30-day measurement window with no exclusions and 30 minutes of downtime.

1. Convert the window to minutes: $30 \times 24 \times 60 = 43{,}200$.
2. Subtract downtime: $43{,}200 - 30 = 43{,}170$ eligible minutes available.
3. Divide and convert to percent: $43{,}170/43{,}200 \times 100 = 99.9306\%$.
4. For a 99.9% target, allowed downtime is $43{,}200(1 - 0.999) = 43.2$ minutes.
5. Remaining budget is $43.2 - 30 = 13.2$ minutes.

The observed result meets this simplified target. Actual agreements may define eligibility, planned maintenance, outage detection, and remedies differently.

Example B: request-based success

Suppose 9,990 of 10,000 eligible requests succeed. The measured success ratio is $9{,}990/10{,}000 = 99.9\%$. This says nothing directly about continuous outage duration: the failures might be concentrated or scattered. Define whether client errors, retries, and incomplete responses enter the denominator.

Example C: latency percentiles

For ten sorted durations $10, 10, 10, 10, 10, 10, 10, 10, 10, 1000$ ms, the mean is $(9 \times 10 + 1000)/10 = 109$ ms. With nearest-rank percentiles, the p95 index is $\lceil 0.95 \times 10 \rceil = 10$, so p95 is 1,000 ms. The p50 index is five, so p50 is 10 ms. A percentile describes the observed sample and is sensitive to sample size and method.

Your turn

1. A 30-day service has 60 minutes of downtime. Does it meet 99.9%?
2. There are 1,980 successful requests out of 2,000. What is the success ratio?
3. A p95 target is 300 ms. A run reports p50 of 80 ms and p95 of 600 ms. Is the latency target met?

Check: (1) 99.8611%, so no; the downtime budget is exceeded by 16.8 minutes. (2) 99%. (3) No; a good median does not establish a good tail.

Terminology: an SLI is the measured value, an SLO is a target, and an SLA states agreed commitments and consequences. These examples teach the engineering vocabulary [R9].

4 Lab 1: setup and your first request

Goal: run one server and establish a reproducible baseline.

1. Extract the supplied ZIP. Open a terminal in the extracted package folder. Enter its `lab` folder. If necessary, use `cd` followed by the actual path in quotes.

```
cd lab
node --version
npm --version
```

2. Check that Node reports version 22 or newer. Then start the server in Terminal A:

```
node app.js
```

   Expected: `Lab PID ...: http://127.0.0.1:3000`. Keep this terminal open.
3. In a browser, open http://127.0.0.1:3000/health. Expected JSON includes `"status":"ok"` and a numeric `pid`. The PID will differ between computers and restarts.
4. Open Terminal B in the same `lab` folder. Run:

```
node load.js http://127.0.0.1:3000/health 100 10
```

   The arguments are the URL, total number of requests, and maximum concurrency. Expect `ok: 100`, `httpErrors: 0`, and `networkErrors: 0`.
5. Save the output with shell redirection if desired:

```
node load.js http://127.0.0.1:3000/health 100 10 > baseline.json
```

   Run one warm-up and three measured repetitions. Record p95 and completed requests/second for each.

Checkpoint: explain why ten concurrent clients do not mean ten Node.js server processes. The client can have ten requests in progress while one server process handles them.

Optional terminal request: use `Invoke-RestMethod` in PowerShell or `curl` in Bash, followed by the health URL. On Windows, `curl.exe` explicitly selects the executable rather than a PowerShell alias. Lab 2 shows both command forms.

Cleanup: Ctrl+C in Terminal A stops the manually launched server. Restart it for the next experiment. All core routes bind to loopback, so they are intended for this local exercise.

5 Reading the server code without getting lost

A request has a short lifecycle

The program imports built-in modules, initialises counters, and creates an HTTP server. When a request arrives, it extracts the pathname and selects a route. The `send` helper sets the HTTP status and JSON content type, then ends the response. An unknown route returns 404. The lab supports GET requests only.

```
if (path === '/health') {
  return send(res, 200, { status: 'ok', pid: process.pid });
}
```

`return` prevents the handler from continuing into another route after responding. In Express, the equivalent response operation would commonly use `res.status(...).json(...)`.

Instrumentation is separate from business behaviour

The server increments the request count on arrival. It increments the error count after a response with status 500 or higher finishes. A snapshot during active traffic can therefore include unfinished requests in the denominator. Wait until the client finishes before the exact-count exercise.

`/metrics` is excluded from request and error totals to keep this classroom measurement easy to interpret. Production systems should document their own inclusion rules. The lab does not implement a production metrics protocol or persistent time-series storage.

Route	Behaviour
`/health`	Returns status and PID
`/metrics`	Returns counters and resource observations
`/fail`	Deliberately returns HTTP 503
`/blocking`	Occupies the event loop for about 100 ms
`/async`	Waits asynchronously for about 100 ms
`/catalogue`	Simulates a 200 ms data read
`/cached`	Uses a five-second local cache
`/counter`	Increments a variable local to this process

Optional request logging

In PowerShell, set `$env:LOG_REQUESTS="1"` before `node app.js`; later remove it with `Remove-Item Env:LOG_REQUESTS`. In Bash, use `LOG_REQUESTS=1 node app.js`. Each log records route, status, duration, and PID. Disable logging for comparisons unless all compared runs use the same setting. Logging itself adds work.

6 Lab 2: metrics and controlled errors

Goal: connect observed behaviour with a clearly defined metric.

1. Stop the server and restart `node app.js`. Do not open `/health` manually during the exact-count exercise.
2. In Terminal B, send exactly 100 successful requests, then ten planned failures:

```
node load.js http://127.0.0.1:3000/health 100 10
node load.js http://127.0.0.1:3000/fail 10 1
```

3. Open http://127.0.0.1:3000/metrics. With no other measured requests, expect `requests: 110`, `errors: 10`, and an error fraction close to `0.090909`. A browser may make an extra favicon request; use the terminal request method below if exact totals differ.
4. Calculate $10/110 \times 100 = 9.09\%$ failed requests and $100/110 \times 100 = 90.91\%$ successful requests for this controlled workload.
5. Inspect `rssMiB`, `heapUsedMiB`, `cpuPercent`, and `loopP99ms`. Record when the snapshot was taken. CPU samples update approximately every second.

```
# PowerShell
Invoke-RestMethod http://127.0.0.1:3000/metrics
# Bash / macOS / Linux
curl http://127.0.0.1:3000/metrics
```

Interpretation: `cpuPercent` divides process CPU time by elapsed wall time and expresses it relative to one CPU core. It can exceed 100% if the process uses several threads. It is not the machine's overall CPU percentage. RSS includes more than the JavaScript heap [R5].

The event-loop delay histogram reports p99 since process startup. It is sampled at 20 ms resolution, so an idle value near 20 ms is not an HTTP response time. Compare it cautiously across equal windows or restart between runs [R4].

Checkpoint: an HTTP 503 is an HTTP error, while connection refusal is a network error in the client. If the process is completely down, it cannot count incoming failures itself. This explains why an external check is useful.

Limitation: the server's simplified error counter only counts completed 5xx responses. The client separately reports non-2xx responses and transport errors. These definitions must be reconciled before using them as one production SLI.

7 Lab 3: observe blocking before changing it

Goal: distinguish CPU blocking from asynchronous I/O waiting.

1. Use one server process. Predict the elapsed time for 100 requests that each occupy the event loop for 100 ms.
2. Run the blocking workload:

   ```
   node load.js http://127.0.0.1:3000/blocking 100 10
   ```

   Expected direction: about ten seconds in total, roughly ten completed requests/second, and queued requests with much larger latency than one isolated request. Hardware and overhead affect the exact values.
3. Run the asynchronous-wait workload with identical arguments:

   ```
   node load.js http://127.0.0.1:3000/async 100 10
   ```

   Expected direction: waits overlap, so the total may be near one second plus overhead and throughput substantially higher.
4. Repeat each three times. Record all errors, mean, p95, and throughput. Inspect metrics during the workload if you want a CPU observation; an idle sample after it finishes may miss the CPU peak.
5. Optional: while `/blocking` is under load, open `/health` from another terminal. Explain why even the trivial route can become slow.

The key difference in code

```
// CPU remains occupied until the loop finishes.
const until = performance.now() + 100;
while (performance.now() < until) {}

// The event loop can handle other work while this waits.
await delay(100);
```

The timer does not perform the busy loop's computation. It models waiting for an external operation. Replacing actual CPU work with a timer would remove the work, not optimise it. For real CPU-heavy calculations, improve the algorithm or use a bounded worker-thread pool or a separate service [R2].

Checkpoint: adding `async` to the busy-loop handler does not make the loop non-blocking. The JavaScript still occupies the same event loop until it yields or completes.

8 Lab 4: measure a cache and its expiry

Goal: explain cold reads, warm reads, and freshness.

1. Restart the server to clear its cache. Run twenty serial uncached reads:

   ```
   node load.js http://127.0.0.1:3000/catalogue 20 1
   ```

   Each read simulates 200 ms of I/O. Expect a total near four seconds plus overhead.
2. Request the cached endpoint once:

   ```
   node load.js http://127.0.0.1:3000/cached 1 1
   ```

   The cache starts empty. This request still pays the simulated data-read delay.
3. Immediately, within five seconds, run:

   ```
   node load.js http://127.0.0.1:3000/cached 20 1
   ```

   The warm requests should be much faster. Do not use a single cold request as your claimed warm-cache result.
4. Wait more than five seconds, then request `/cached` again. Expect a refresh delay. Use the browser or terminal request command to compare `source` and `generatedAt`.
5. Restart, then send ten concurrent requests to `/cached`. The shared `pending` promise lets concurrent misses reuse one in-flight data read in this process.

Why the cache remains bounded

This implementation stores one catalogue object, not an entry for every URL or user. A general cache needs capacity limits and eviction. Expiry alone is insufficient if expired keys remain allocated indefinitely.

Worked improvement calculation

Illustrative mean latency: before 205 ms, warm cache 5 ms. Reduction is $(205 - 5)/205 \times 100 = 97.56\%$. The speed ratio is $205/5 = 41$. This conclusion applies to the stated warm-cache workload. It says nothing about cold misses, changing data, or a different application.

Checkpoint: a five-second TTL permits a cached snapshot to be served for five seconds after filling. Decide whether that freshness is acceptable. Ticket availability and completed purchases may require stronger consistency than a public product description.

9 Lab 5: two workers, separate state

Goal: see that cluster workers are separate processes.

1. Stop `app.js` with Ctrl+C. Start the supplied cluster launcher in Terminal A:

   ```
   node cluster.js
   ```

   Expect one primary-process message and two worker listening messages. Workers share the listening port through Node.js cluster [R3].
2. In Terminal B, run:

   ```
   node inspect-workers.js
   ```

   The script makes twenty requests with fresh connections. Look for different PIDs and increasing counts within each PID. Request distribution depends on the platform and scheduling; repeat if one worker dominates a small sample.
3. Re-run the blocking experiment:

   ```
   node load.js http://127.0.0.1:3000/blocking 100 10
   ```

   Compare with the single-process result using the same workload. Two available cores may allow roughly twice the CPU work in parallel, but overhead, CPU quotas, and client limits can reduce the improvement.
4. Request `/metrics` repeatedly. Each response describes the worker that answered, not the entire cluster. A full monitoring system must aggregate appropriately across workers.
5. Stop the cluster with Ctrl+C before moving to PM2.

A prediction worth testing

If worker A reports count 6 and worker B reports count 4, the next count is not necessarily 11. Each worker owns its own counter. The same principle applies to in-memory sessions and caches. The process that handles a request may have never seen that user's previous request.

A more scalable design

Store durable registration data in a database and shared session data in a suitable shared store. Keep workers replaceable. A local cache can still be useful if separate copies and occasional staleness are acceptable. Use atomic operations or transactions for shared counters that must remain correct under concurrent updates.

Checkpoint: killing the host would remove both workers. This lab demonstrates process-level scaling on one machine, not high availability across machines. The teaching launcher respawns a failed worker, but a production supervisor also needs restart limits and operational visibility.

10 Lab 6: optional PM2 management

Goal: use a process manager instead of maintaining a manual launcher. This extension requires npm network access. Run it in the same `lab` folder, after stopping all manually launched lab servers.

1. Install PM2 locally and record its version:

   ```
   npm install --save-dev pm2
   npx pm2 --version
   ```

   Keep the generated package lock if you need to reproduce the installed dependency version. The authoring environment blocked PM2's local control socket, so this optional workflow was documentation-checked but could not be fully runtime-tested there.
2. Start two managed workers directly from `app.js`. Do not run `cluster.js` inside PM2 cluster mode.

   ```
   npx pm2 start app.js --name comp3028-lab -i 2
   npx pm2 status
   npx pm2 monit
   ```

   Expect two online instances. Ctrl+C exits the monitor while PM2 continues managing the application.
3. Inspect distribution and run your comparison:

   ```
   node inspect-workers.js
   node load.js http://127.0.0.1:3000/blocking 100 10
   ```

4. Reload, inspect status, and verify the health endpoint:

   ```
   npx pm2 reload comp3028-lab
   npx pm2 status
   npx pm2 logs comp3028-lab --lines 20
   ```

   Ctrl+C leaves the log viewer. A reload replaces workers; PIDs may change. The app handles shutdown signals and allows a short drain period.
5. Clean up only this lab application:

   ```
   npx pm2 delete comp3028-lab
   ```

PM2 can supervise and inspect processes, but it does not guarantee uninterrupted service during every failure or deployment. Reboot persistence is separate and platform-specific. This lab does not install a startup service. Consult the official PM2 documentation for the target operating system [R7, R8].

If PM2 cannot be installed: complete Lab 5 using Node.js cluster. It covers the process and memory concepts without third-party dependencies. Do not silently treat the optional PM2 extension as completed.

11 Beyond the lab: queries, pools, and multiple hosts

A query can become the bottleneck

Suppose a registration page loads all rows and filters them in JavaScript. First select only the needed records and columns. For a relational database, a query such as the following gives the database an opportunity to filter efficiently:

```
SELECT event_id, title
FROM events
WHERE campus_id = 7
ORDER BY event_id
LIMIT 20;
```

An index involving `campus_id` and `event_id` might help this access pattern, depending on data size, distribution, and the planner. Inspect a representative plan with `EXPLAIN`; use `EXPLAIN ANALYZE` thoughtfully because it executes the statement [R10]. This is a conceptual extension, not part of the dependency-free lab.

Connection pools multiply with workers

If one worker can open ten database connections and you start eight workers, the application can potentially open 80 connections. Add other applications and administrative headroom before selecting limits. If the database supports only 50 connections, merely adding workers can cause failures. Pool waiting time is often more informative than application CPU alone.

A reverse proxy connects multiple instances

A reverse proxy receives client traffic and forwards it to selected backends. Across two hosts, it can route to an available instance while one is unavailable, provided the rest of the design supports failure handling. The proxy, database, network, and shared state each need their own resilience plan. NGINX supports multiple load-balancing methods [R11].

Memory investigation, step by step

1. Reproduce comparable workloads and record memory over time.
2. Allow similar idle periods and inspect whether the baseline keeps rising.
3. Check unbounded data structures, listeners, timers, and cache keys.
4. In a suitable test environment, compare heap snapshots and retained objects.
5. Remove the retention cause, repeat the workload, and verify behaviour.

A single high RSS snapshot does not establish a leak. Collecting heap snapshots can pause the process and consume memory, so use an appropriate test environment.

12 Troubleshooting and measurement discipline

Symptom	Check and recovery
`node` not found	Install a supported Node.js release, reopen the terminal, and check `node -version`.
Module/file not found	Confirm the terminal is in the extracted `lab` folder. Use `dir` in PowerShell or `ls` in Bash.
`EADDRINUSE`	Port 3000 is occupied. Stop your previous lab server or delete the named PM2 lab app. Do not terminate unrelated processes.
Connection refused	Check Terminal A, the port, and whether the server exited. The client must use HTTP and loopback.
Unexpected 404	Check the endpoint spelling. Browser favicon requests can also produce 404.
Warm cache seems slow	The TTL may have expired or another worker may have a cold cache. Start with a single process.
Two workers not faster	Check available CPU capacity, CPU quotas, workload type, errors, and client saturation.
PM2 remains online	Ctrl+C exits the viewer, not the daemon-managed app. Delete `comp3028-lab` for cleanup.

Using another port

For the single-process lab, PowerShell: `$env:PORT="3001"`, then `node app.js`. Bash: `PORT=3001 node app.js`. Replace port 3000 with 3001 in client URLs. The worker-inspection helper is intentionally fixed to 3000; edit its URL if using it on another port. Remove the PowerShell variable with `Remove-Item Env:PORT` afterwards.

What the client actually measures

`load.js` sends a fixed number of requests at bounded concurrency. It uses a fresh HTTP connection for each request, drains the response, and measures end-to-end duration in the client. It counts 2xx as success, other HTTP statuses as HTTP errors, and connection/timeout failures separately. Percentiles include all completed attempts, including failures. The ten-second timeout is an inactivity timeout, not a strict total request deadline.

The client reports completed attempts per second, not guaranteed successful business transactions per second. A fixed-concurrency test can reduce its offered rate when the server slows. It is a learning tool rather than a production capacity certification.

13 Evidence worksheet and final case study

Record the environment

Node version: _____ Operating system: _____

Available CPU / quota: _____ Logging enabled? _____

Record each measured run

Use the same workload within each before/after comparison. Repeat this table three times, or make a spreadsheet with one row per run.

Configuration	Requests	Concurrency	p95 ms	Completed/s
Single / blocking	100	10		
Single / async wait	100	10		
Uncached catalogue	20	1		
Warm cached catalogue	20	1		
Two workers / blocking	100	10		

Also record successful requests, HTTP errors, and network errors. Include cold-cache results separately. Use the median of three runs as a compact summary and retain the individual outputs.

Final case: registration under pressure

The event-registration system has four application workers. At 09:05, catalogue p95 is 1.8 seconds, CPU is 25%, database connections are saturated, and request failures are 3%. The team proposes doubling the worker count.

1. Explain why this proposal might make the problem worse.
2. Identify three measurements that would narrow the diagnosis.
3. Propose one change with a freshness or correctness trade-off.
4. State an acceptance test that includes performance and reliability.
5. Identify which state must survive worker replacement.

Suggested answer: extra workers may multiply connection demand. Inspect query duration, pool wait, and active connections. Test query improvements or a bounded catalogue cache with an agreed TTL. For a hypothetical target, require correct registrations, at least 99.9% successful eligible requests, and p95 below 500 ms at the stated workload. Durable registrations belong in a database; sessions need an appropriate shared design.

Suggested submission: environment details, commands, raw output, comparison table, one explanation of the bottleneck, one trade-off, and evidence that the service was cleaned up. Check the current LMS for the actual assessment requirements.

14 References and scope of the rewrite

The supplied lecture's topic sequence is retained. The rewrite adds definitions, calculations, a local application, guided experiments, interpretation, and troubleshooting. It corrects the distinction between cluster processes and threads and avoids treating an ORM or extra workers as an automatic performance improvement.

The supplied deck's project-contribution percentages appear in the slides as source-specific information. Submission dates, GitHub access behaviour, and the final individual-mark formula require the current course instructions. The supplied deck does not provide enough information to reconstruct that final formula.

The historical vendor-policy examples have been replaced by explicit classroom service examples. No provider's current contractual uptime promise is asserted here.

1. Original teaching material. Supplied COMP3028 Lecture 10: Optimising Server Management, undated PDF. Rewritten and expanded for beginners.
2. Node.js: event loop. https://nodejs.org/en/learn/asynchronous-work/dont-block-the-event-loop
3. Node.js: cluster. https://nodejs.org/api/cluster.html
4. Node.js: measurement. https://nodejs.org/api/perf_hooks.html
5. Node.js: process metrics. https://nodejs.org/api/process.html
6. Node.js: HTTP. https://nodejs.org/api/http.html
7. PM2: quick start. https://pm2.keymetrics.io/docs/usage/quick-start/
8. PM2: cluster mode. https://pm2.keymetrics.io/docs/usage/cluster-mode/
9. Google SRE: service-level objectives. https://sre.google/sre-book/service-level-objectives/
10. PostgreSQL: EXPLAIN. https://www.postgresql.org/docs/current/using-explain.html
11. NGINX: load balancing. https://nginx.org/en/docs/http/load_balancing.html

Official technical sources consulted 10 November 2026. Reference labels R1–R11 follow the numbered list. The complete lab is supplied for local teaching use. Its synthetic routes, local counters, and short workload generator are intentionally simpler than a production deployment.

A Complete runnable source

The files below are identical to those in the supplied lab folder. Run the files directly to avoid transcription errors. The code uses built-in Node.js modules and CommonJS imports.

A.1 app.js

```
'use strict';
// Local teaching server. Intentionally includes one blocking route.
const http = require('node:http');
const { performance, monitorEventLoopDelay } = require('node:perf_hooks');
const { setTimeout: delay } = require('node:timers/promises');
const port = Number(process.env.PORT || 3000);
const histogram = monitorEventLoopDelay({ resolution: 20 });
histogram.enable();
let requests = 0, errors = 0, counter = 0;
let cache = null, pending = null;
let previousCpu = process.cpuUsage(), previousTime = performance.now();
let cpuPercent = 0;
const sampler = setInterval(() => {
  const now = performance.now();
  const used = process.cpuUsage(previousCpu);
  cpuPercent = 100 * (used.user + used.system) / ((now - previousTime) * 1000);
  previousCpu = process.cpuUsage(); previousTime = now;
}, 1000);
sampler.unref();
function send(res, status, value) {
  res.writeHead(status, { 'Content-Type': 'application/json' });
  res.end(JSON.stringify(value));
}
async function catalogue() {
  await delay(200); // Simulated I/O, not a real database.
  return { items: ['Notebook', 'Pen'], generatedAt: new Date().toISOString() };
}
const server = http.createServer(async (req, res) => {
  const started = performance.now();
  const path = new URL(req.url, 'http://localhost').pathname;
  const measured = path !== '/metrics';
  if (measured) requests++;
  res.on('finish', () => {
    if (measured && res.statusCode >= 500) errors++;
    if (measured && process.env.LOG_REQUESTS === '1') {
      console.log(JSON.stringify({ path, status: res.statusCode,
        ms: +(performance.now() - started).toFixed(2), pid: process.pid }));
    }
  });
  try {
    if (req.method !== 'GET') return send(res, 405, { error: 'GET only' });
    if (path === '/health') return send(res, 200, { status: 'ok', pid: process.pid });
    if (path === '/fail') return send(res, 503, { error: 'Planned lab failure' });
    if (path === '/counter') return send(res, 200, { pid: process.pid, count: ++counter });
    if (path === '/blocking') {
      const until = performance.now() + 100;
      while (performance.now() < until) {} // Deliberate CPU blocking.
      return send(res, 200, { mode: 'blocking', pid: process.pid });
    }
    if (path === '/async') {
      await delay(100); // Models waiting for I/O, NOT CPU work offloaded.
      return send(res, 200, { mode: 'async-wait', pid: process.pid });
    }
    if (path === '/catalogue') return send(res, 200, await catalogue());
    if (path === '/cached') {
```

```
      if (cache && Date.now() < cache.expires) {
        return send(res, 200, { source: 'hit', ...cache.value });
      }
      // Share an in-flight read so concurrent misses do not duplicate work.
      if (!pending) pending = catalogue().then(value => {
        cache = { value, expires: Date.now() + 5000 };
        return value;
      }).finally(() => { pending = null; });
      return send(res, 200, { source: 'miss-or-coalesced', ...await pending });
    }
    if (path === '/metrics') {
      const memory = process.memoryUsage();
      return send(res, 200, { pid: process.pid, requests, errors,
        errorRate: requests ? errors / requests : 0,
        cpuPercent: +cpuPercent.toFixed(1),
        rssMiB: +(memory.rss / 1048576).toFixed(1),
        heapUsedMiB: +(memory.heapUsed / 1048576).toFixed(1),
        loopP99ms: +(histogram.percentile(99) / 1e6).toFixed(2) });
    }
    send(res, 404, { error: 'Route not found' });
  } catch (error) {
    console.error(error.message);
    if (!res.headersSent) send(res, 500, { error: 'Internal error' });
    else res.destroy();
  }
});
server.listen(port, '127.0.0.1', () => console.log(`Lab PID ${process.pid}: http://127.0.0.1:${
    port}`));
function shutdown() {
  server.close(() => { histogram.disable(); clearInterval(sampler); process.exit(0); });
  setTimeout(() => process.exit(1), 5000).unref();
}
process.on('SIGINT', shutdown);
process.on('SIGTERM', shutdown);
```

A.2 load.js

```
'use strict';
// Fixed request count, fixed concurrency, local destinations only.
const http = require('node:http');
const { performance } = require('node:perf_hooks');
const target = new URL(process.argv[2] || 'http://127.0.0.1:3000/health');
const total = Number(process.argv[3] || 100);
const concurrency = Number(process.argv[4] || 10);
if (!['127.0.0.1', 'localhost'].includes(target.hostname) || target.protocol !== 'http:') {
  throw new Error('This classroom client accepts local HTTP targets only.');
}
if (!Number.isInteger(total) || total < 1 || total > 10000 ||
    !Number.isInteger(concurrency) || concurrency < 1 || concurrency > 100) {
  throw new Error('Use 1..10000 requests and 1..100 concurrent clients.');
}
let next = 0, ok = 0, httpErrors = 0, networkErrors = 0;
const latencies = [];
function request() {
  return new Promise(resolve => {
    const start = performance.now();
    let settled = false;
    const finish = success => {
      if (settled) return;
      settled = true;
      latencies.push(performance.now() - start);
      if (success === null) networkErrors++;
```

```
      else if (success) ok++;
      else httpErrors++;
      resolve();
    };
    // Fresh connections also make worker distribution easier to observe.
    const req = http.get(target, { agent: false }, res => {
      res.resume();
      res.on('end', () => finish(res.statusCode >= 200 && res.statusCode < 300));
      res.on('error', () => finish(null));
    });
    req.setTimeout(10000, () => req.destroy(new Error('timeout')));
    req.on('error', () => finish(null));
  });
}
async function worker() { while (next++ < total) await request(); }
(async () => {
  const start = performance.now();
  await Promise.all(Array.from({ length: Math.min(concurrency, total) }, worker));
  const seconds = (performance.now() - start) / 1000;
  latencies.sort((a,b) => a-b);
  const percentile = p => latencies[Math.ceil(p * latencies.length) - 1];
  console.log(JSON.stringify({ total, concurrency, ok, httpErrors, networkErrors,
    seconds: +seconds.toFixed(3), completedPerSecond: +(total / seconds).toFixed(1),
    meanMs: +(latencies.reduce((a,b) => a+b, 0) / total).toFixed(2),
    p50ms: +percentile(.5).toFixed(2), p95ms: +percentile(.95).toFixed(2),
    p99ms: +percentile(.99).toFixed(2) }, null, 2));
})();
```

A.3 cluster.js

```
'use strict';
const cluster = require('node:cluster');
if (cluster.isPrimary) {
  console.log(`Primary PID ${process.pid}; starting two workers`);
  let stopping = false;
  for (let i = 0; i < 2; i++) cluster.fork();
  cluster.on('exit', worker => {
    console.log(`Worker ${worker.process.pid} exited`);
    if (!stopping) cluster.fork();
  });
  const stop = () => {
    if (stopping) return;
    stopping = true;
    for (const worker of Object.values(cluster.workers)) worker.process.kill('SIGTERM');
  };
  process.on('SIGINT', stop);
  process.on('SIGTERM', stop);
} else {
  require('./app');
}
```

A.4 inspect–workers.js

```
'use strict';
const http = require('node:http');
const seen = new Set();
async function once() {
  await new Promise((resolve, reject) => {
    const req = http.get('http://127.0.0.1:3000/counter', {agent: false}, res => {
```

```
      let body = '';
      res.on('data', part => { body += part; });
      res.on('end', () => { const data = JSON.parse(body); seen.add(data.pid); console.log(data);
     resolve(); });
      res.on('error', reject);
    });
    req.setTimeout(5000, () => req.destroy(new Error('timeout')));
    req.on('error', reject);
  });
}
(async () => {
  for (let i = 0; i < 20; i++) await once();
  console.log('Distinct worker PIDs:', [...seen]);
})().catch(e => { console.error(e.message); process.exitCode = 1; });
```